El método simplex de la teoría al código con Python

Maximización y minimización, restricciones con ≤, ≥ y =, la técnica Big-M y cada paso del algoritmo traducido al código real del proyecto

Proyecto metodo-simplex — aplicación de escritorio (Tkinter)

Contenido

1. El problema que resuelve el método 3
2. La idea geométrica: de esquina en esquina 3
3. Forma estándar: holguras, superávit y Big-M 4
4. La fila de costos reducidos (c■ – z■) 4
5. El ciclo: escoje, cociente y pivote 5
    - 5.1 Variable que entra . . . 5
    - 5.2 El que sale: test del cociente . . . 5
    - 5.3 El pivote (eliminación de Gauss-Jordan) . . . 5
    - 5.4 El bucle que lo coordina . . . 6
6. Cómo decide el método que terminó 6
7. Minimización sin reescribir el método 7
8. Ejemplo numérico completo 7
    - 8.1 Paso 0 — la tabla inicial . . . 7
    - 8.2 Iteración 1 — entra x■ . . . 7
    - 8.3 Iteración 2 — entra x■ . . . 7
    - 8.4 En gráficas . . . 8
9. Las facilidades de Python que hicieron esto simple 9
10. Para cerrar: cómo usar el proyecto 10

1. El problema que resuelve el método

Un problema de **programación lineal** pide optimizar (maximizar o minimizar) una *función objetivo* lineal $Z = c_1 \cdot x_1 + c_2 \cdot x_2 + \ldots + c_n \cdot x_n$, sujeta a un conjunto de *restricciones* lineales y a que las variables de decisión sean no negativas:

$$\begin{aligned} \text{Maximizar } Z &= 3 \cdot x_1 + 2 \cdot x_2 \\ \text{sujeto a } & x_1 + x_2 \leq 4 \\ & 2 \cdot x_1 + x_2 \leq 5 \\ & x_1,\ x_2 \geq 0 \end{aligned}$$

Este formato se llama **forma general**. En el proyecto ese modelo se escribe con una *dataclass* de Python, una de las facilidades que usamos y que veremos en detalle en el capítulo 9:

```
@dataclass(frozen=True)
class LPProblem:
    objective_sense: str
    objective_coeffs: tuple[float, ...]
    constraints: tuple[Constraint, ...]

    def validate(self) -> None:
        if self.objective_sense not in VALID_SENSES:
            raise ValueError("El sentido del objetivo debe ser 'max' o 'min'.")
        if not self.objective_coeffs:
            raise ValueError("Debe haber al menos una variable.")
        if not self.constraints:
            raise ValueError("Debe haber al menos una restricción.")
        expected = len(self.objective_coeffs)
        for constraint in self.constraints:
            if constraint.num_variables != expected:
                raise ValueError(
                    "Todas las restricciones deben tener la misma cantidad "
                    "de coeficientes que el objetivo."
                )
            constraint.validate()

    @property
    def num_variables(self) -> int:
        return len(self.objective_coeffs)

    @property
    def num_constraints(self) -> int:
        return len(self.constraints)
```

La dataclass declara *qué es* un problema, no *cómo* se resuelve. Con **frozen=True** la hacemos inmutable: un problema no puede mutarse a mitad de la resolución, lo que evita errores sutiles al pivotar. Cada restricción guarda sus coeficientes, la relación (≤, ≥, =) y el término independiente. El método simplex toma esta estructura y hace lo que describimos a continuación.

2. La idea geométrica: de esquina en esquina

Cada restricción corta el plano en dos: un lado permitido y otro prohibido. La intersección de todos los lados permitidos (incluyendo $x \geq 0$) es la **región factible**, que siempre es un polígono convexo. Los **vértices** (esquinas) son los puntos donde varias restricciones se cumplen con igualdad.

Una propiedad fundamental de la programación lineal dice que si existe una solución óptima, hay una en algún **vértice**. El simplex explota esa propiedad: en vez de probar puntos al azar, **visita vértices vecinos de uno en uno, mejorando Z en cada paso**, y se detiene cuando ningún vecino mejora la solución.

Concretamente el método opera sobre una **tabla simplex** (tableau), una hoja de cálculo donde cada fila es una ecuación y cada columna una variable. De un vértice a otro se pasa con un **pivote**: una variable **entra** a la base (deja de valer 0) y otra **sale**. La base es el conjunto de variables activas que definen el vértice actual.

El código del proyecto guarda esa lógica en variables con nombre —*entering*, *leaving*, *pivot_row*, *pivot_col*— y **registra cada iteración como un paso** para poder mostrarla y explicarla en la aplicación:

```
@dataclass
class SimplexStep:
    kind: str          # 'build' | 'iterate' | 'optimal' | 'unbounded' | 'infeasible'
    matrix: ...        # tabla simplex completa
    entering: str | None
    leaving: str | None
    pivot_row: int | None
    pivot_col: int | None
    ratios: tuple[float | None, ...]
    ...
```

Esta estructura es lo que alimenta la pestaña 'Paso a paso' de la interfaz: el tableau, el cociente mínimo y el intercambio de variables de cada vértice visitado.

3. Forma estándar: holguras, superávit y Big-M

La tabla simplex solo sabe de *ecuaciones* (=), no de desigualdades. Por eso el primer paso es convertir cada restricción en una igualdad agregando variables auxiliares:

- $x_1 + x_2 \leq 4 \rightarrow x_1 + x_2 + s_1 = 4$ (s_1 = holgura, mide lo que sobra, costo 0)
- $x_1 + x_2 \geq 3 \rightarrow x_1 + x_2 - e_1 + a_1 = 3$ (e_1 = superávit, costo 0; a_1 = artificial)
- $x_1 + x_2 = 5 \rightarrow x_1 + x_2 + a_2 = 5$ (a_2 = artificial)

Holguras y superávits son variables honestas: representan 'cuánto sobra' o 'cuánto se pasa'. Las **artificiales** son un truco de la técnica **Big-M**: se agregan solo para que las filas ≥ y = tengan una variable natural con la que empezar la base (una columna unitaria de identidad). Para expulsarlas, llevan un costo enorme M en el objetivo, tan grande que el método las saca de la base solas en cuanto puede.

El código hace exactamente esto en **_build_canonical_form**. Lo esencial es el bucle que recorre las restricciones y coloca 1, −1 o 1 según la relación:

```
    for row, constraint in enumerate(problem.constraints):
        matrix[row, :num_variables] = constraint.coefficients
        relation = constraint.relation
        if relation == "<=":
            matrix[row, slack_index] = 1.0
            basis_names.append(slack_labels[slack_count])
            basis_indices.append(slack_index)
            slack_index += 1
            slack_count += 1
        elif relation == ">=":
            matrix[row, surplus_index] = -1.0
            surplus_index += 1
            surplus_count += 1
            matrix[row, artificial_index] = 1.0
            label = artificial_labels_list[artificial_count]
            artificial_labels.add(label)
            basis_names.append(label)
            basis_indices.append(artificial_index)
            artificial_index += 1
            artificial_count += 1
```

Observa que la **base inicial** queda formada por holguras (en ≤) y artificiales (en ≥ y =). El costo −M de las artificiales se define en la línea `costs[...] = -big_m`, y el valor numérico de M se calcula con una heurística basada en la magnitud de los datos (capítulo 9).

4. La fila de costos reducidos ($c_j - z_j$)

Para saber si conviene traer una variable a la base, el método calcula, para cada variable no básica x_j, cuánto cambiaría Z si metiéramos una unidad de x_j. Eso es el **costo reducido** $c_j - z_j$:

```
z_j = c_B · (columna j) → costo 'oculto' de x_j por las variables básicas que desplaza
                    c_j − z_j → beneficio neto por unidad de x_j
          En maximización: si c_j − z_j > 0, x_j mejora Z → candidata a entrar.
```

El código lo implementa en tres líneas gracias al **producto punto de NumPy** (el @ multiplica matrices y vectores completos a la vez):

```
def _build_objective_row(canonical: CanonicalForm) -> np.ndarray:
    basic_costs = canonical.costs[canonical.basis_indices]
    implied_costs = basic_costs @ canonical.matrix
    reduced_row = implied_costs - canonical.costs
    objective_value = float(basic_costs @ canonical.rhs)
    return np.append(reduced_row, objective_value)
```

Internamente la fila se guarda como $z_j - c_j$: así el criterio de parada es natural (óptimo cuando todos los valores son ≥ 0). Al mostrar al usuario la negamos para que se lea como $c_j - z_j$, la forma clásica de los libros:

```
reduced_costs = tuple(-(objective_row))
```

5. El ciclo: escoje, cociente y pivote

El bucle del simplex repite tres decisiones hasta no poder mejorar:

5.1 Variable que entra

Entre las no básicas, se elige la de mejor costo reducido. Como trabajamos con la fila $z_j - c_j$, 'mejor' significa el valor **más negativo**. Si ninguna es negativa, estamos en el óptimo. El código excluye primero las variables básicas (por ruido numérico puede aparecer un −0.0000000001) y, entre empates, gana el índice más pequeño: la **regla de Bland**, que evita ciclos infinitos en problemas degenerados.

```
def _pick_entering_column(canonical: CanonicalForm, objective_row: np.ndarray) -> int | None:
    nonbasic = [i for i in range(canonical.num_columns) if i not in set(canonical.basis_indices)]
    candidates = [i for i in nonbasic if objective_row[i] < -NEGLIGIBLE]
    if not candidates:
        return None
    return min(candidates, key=lambda i: (objective_row[i], i))
```

5.2 El que sale: test del cociente

No basta con elegir quién entra; hay que saber quién sale para que el nuevo vértice siga siendo factible. Para cada fila con coeficiente positivo en la columna entrante, se divide *el lado derecho entre ese coeficiente*; el mínimo de esos cocientes es la restricción que se agota primero, y esa es la fila que debe salir. Si **ninguna** fila tiene coeficiente positivo, x_j puede crecer sin límite y el problema es **no acotado**.

```
def _ratio_test(canonical: CanonicalForm, matrix: np.ndarray, entering_col: int) -> tuple[list[float | None], int | None]
    ratios: list[float | None] = []
    best_row: int | None = None
    best_ratio = float("inf")
    for row in range(canonical.num_rows):
        coefficient = matrix[row, entering_col]
        rhs_value = matrix[row, -1]
        if coefficient > NEGLIGIBLE:
            ratio = rhs_value / coefficient
            ratios.append(ratio)
            if ratio < best_ratio:
                best_ratio = ratio
                best_row = row
        else:
            ratios.append(None)
    return ratios, best_row
```

5.3 El pivote (eliminación de Gauss-Jordan)

El pivote normaliza la fila elegida y la usa para limpiar su columna en todas las demás filas, **incluida la del objetivo**. Como la fila del objetivo participa del pivote, el valor de Z en la última columna se actualiza solo, sin

recalcular nada aparte.

```
def _perform_pivot(matrix: np.ndarray, pivot_row: int, pivot_col: int) -> None:
    pivot_value = matrix[pivot_row, pivot_col]
    matrix[pivot_row] = matrix[pivot_row] / pivot_value
    for row in range(matrix.shape[0]):
        if row == pivot_row:
            continue
        factor = matrix[row, pivot_col]
        if abs(factor) > NEGLIGIBLE:
            matrix[row] = matrix[row] - factor * matrix[pivot_row]
```

5.4 El bucle que lo coordina

Todo esto se coordina en solve() con un bucle de hasta MAX_PIVOTS pasadas (cortafuegos contra un ciclo degenerado casi imposible). Cada iteración se guarda con su tableau, ratios y pivote para mostrarlos después:

```
        for _ in range(MAX_PIVOTS):
            objective_row = tableau[-1, :-1]
            entering_col = _pick_entering_column(canonical, objective_row)
            if entering_col is None:
                break
            entering_name = canonical.column_labels[entering_col]
            ratios, leaving_row = _ratio_test(canonical, tableau, entering_col)
            if leaving_row is None:
                steps.append(
                    self._make_step(
                        canonical, tableau, kind="unbounded", ratios=ratios,
                        entering=entering_name, leaving=None, pivot_row=None,
                        pivot_col=entering_col, note="",
                    )
                )
                status = "unbounded"
                break
            leaving_name = canonical.basis_names[leaving_row]
            pivot_value = tableau[leaving_row, entering_col]
            steps.append(
                self._make_step(
                    canonical, tableau, kind="iterate", ratios=ratios,
                    entering=entering_name, leaving=leaving_name,
                    pivot_row=leaving_row, pivot_col=entering_col, note="",
                )
            )
            _perform_pivot(tableau, leaving_row, entering_col)
            canonical.basis_names[leaving_row] = entering_name
            canonical.basis_indices[leaving_row] = entering_col
            num_pivots += 1
```

6. Cómo decide el método que terminó

El simplex siempre llega a un estado donde ninguna variable no básica mejora la solución, pero eso no siempre significa éxito. Hay tres desenlaces posibles:

- **Óptimo:** ningún costo reducido es favorable (todo $z_j - c_j \geq 0$). El valor de la última celda es el Z óptimo.
- **No acotado:** hay variable entrante, pero ninguna fila la limita (el cociente no existe).
- **Infactible:** terminó con una variable artificial todavía en la base y con valor positivo.

El caso infactible es la sutileza real del Big-M: la técnica artificial funciona, pero si las restricciones son contradictorias, la artificial 'no tiene salida' y queda en la base. Por eso la señal no es la presencia de la artificial, sino su valor:

```
def _artificial_remains_in_basis(canonical: CanonicalForm, matrix: np.ndarray) -> bool:
    return any(
        canonical.column_labels[col] in canonical.artificial_labels
        and matrix[row, -1] > INFEASIBLE_TOLERANCE
        for row, col in enumerate(canonical.basis_indices)
    )
```

El 1e-6 indica 'valor efectivamente distinto de cero': si la artificial quedó con valor 0, la restricción se cumple al límite (degeneración legítima) y el problema sí es factible.

7. Minimización sin reescribir el método

El simplex descrito maximiza. Pero minimizar Z es lo mismo que maximizar −Z, así que el programa niega el vector de costos c de la función objetivo y resuelve un problema máximo equivalente. Al reportar el resultado, vuelve a negar el valor interno para entregar el Z de tu problema original.

```
costs_max = np.array(problem.objective_coeffs, dtype=float)
if problem.objective_sense == 'min':
    costs_max = -costs_max    # minimizar Z == maximizar -Z
```

```
def _display_objective(canonical: CanonicalForm, internal_value: float) -> float:
    return internal_value if canonical.is_max_form else -internal_value
```

8. Ejemplo numérico completo

Resolvamos con nuestro propio código el problema de la portada: **maximizar Z = 3·x_1 + 2·x_2** con $x_1 + x_2 \le 4$ y $2 \cdot x_1 + x_2 \le 5$, $x \ge 0$.

Las cifras de este capítulo no están escritas a mano: se generan ejecutando BigMSolver en vivo mientras se crea el PDF. Lo que ves abajo es la salida real del algoritmo.

8.1 Paso 0 — la tabla inicial

El programa convierte cada ≤ en igualdad con su holgura (s_1 y s_2) y arma el tableau inicial. La última fila muestra los costos reducidos $c_j - z_j$; la base inicial es (s_1, s_2) con Z = 0. La columna del lado derecho (b) son los términos independientes.

	x1	x2	s1	s2	b
s1	1	1	1	0	4
s2	2	1	0	1	5
Z	**3**	**2**	**0**	**0**	**0**

En la fila de costos reducidos hay valores positivos (3 para x_1 y 2 para x_2): hay espacio para mejorar, así que el ciclo arranca.

8.2 Iteración 1 — entra x_1

x_1 tiene el costo reducido más alto (3): cada unidad que entra sube Z en 3. ¿Qué restricción lo limita? Aplicamos el test del cociente con la columna de x_1:

```
s1: 4 / 1 = 4 s2: 5 / 2 = 2.5 → sale s2 (cociente mínimo 2.5)
```

s_2 da el cociente mínimo (2.5), así que s_2 sale de la base y x_1 entra. El pivote es el elemento (fila s_2, columna x_1) = 2. Tras el pivote, la base pasa a ser (s_1, x_1):

	x1	x2	s1	s2	b	b/a
s1	0	0.5	1	-0.5	1.5	3
x1	1	0.5	0	0.5	2.5	5
Z	**0**	**0.5**	**0**	**-1.5**	**7.5**	

Ahora x_1 vale 2.5 (su lado derecho) y Z subió a 7.5. Pero la fila de costos reducidos todavía tiene un valor positivo (0.5 en x_2), así que podemos mejorar de nuevo.

8.3 Iteración 2 — entra x_2

Sobre la tabla nueva, entra x_2 (costo reducido +0.5). El cociente da 3 en s_1 y 5 en la fila de x_1, así que sale s_1; el pivote es 0.5.

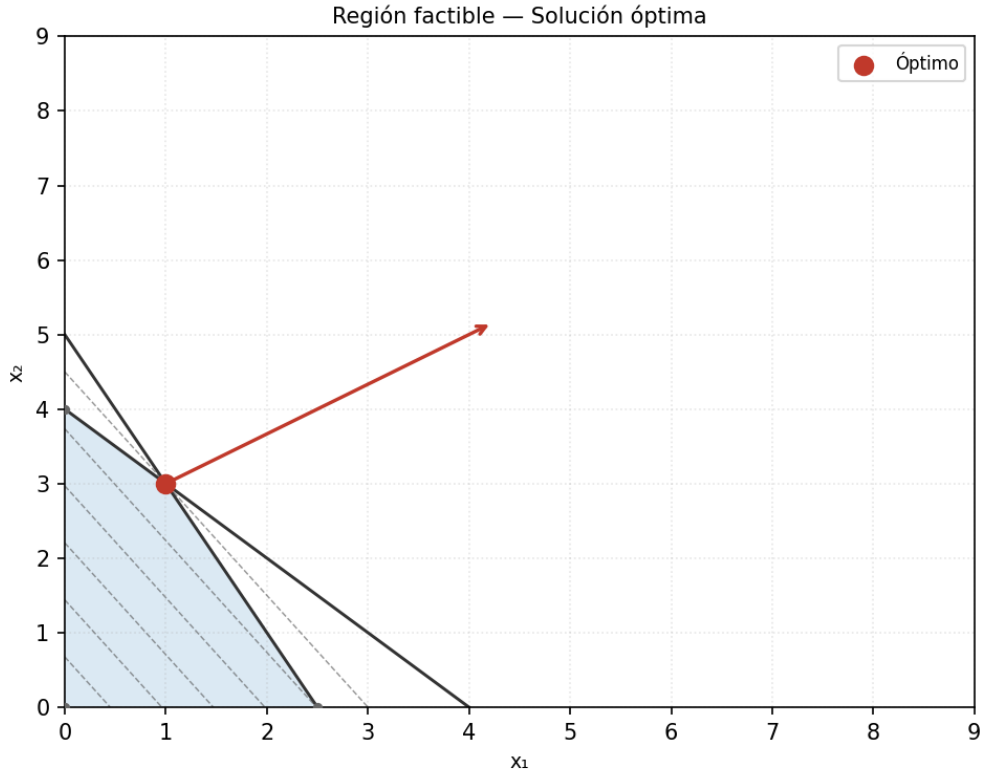
```
s1: 1.5 / 0.5 = 3 x1: 2.5 / 0.5 = 5 → sale s1 (cociente mínimo 3)
```

	x1	x2	s1	s2	b
x2	0	1	2	-1	3
x1	1	0	-1	1	1
Z	**0**	**0**	**-1**	**-1**	**9**

La fila de costos reducidos ya no tiene valores positivos: **óptimo alcanzado**. Solución: $x_1 = 1$, $x_2 = 3$, con Z = 9. Comprobación: 1 + 3 = 4 (límite justo) y 2·1 + 3 = 5 (límite justo).

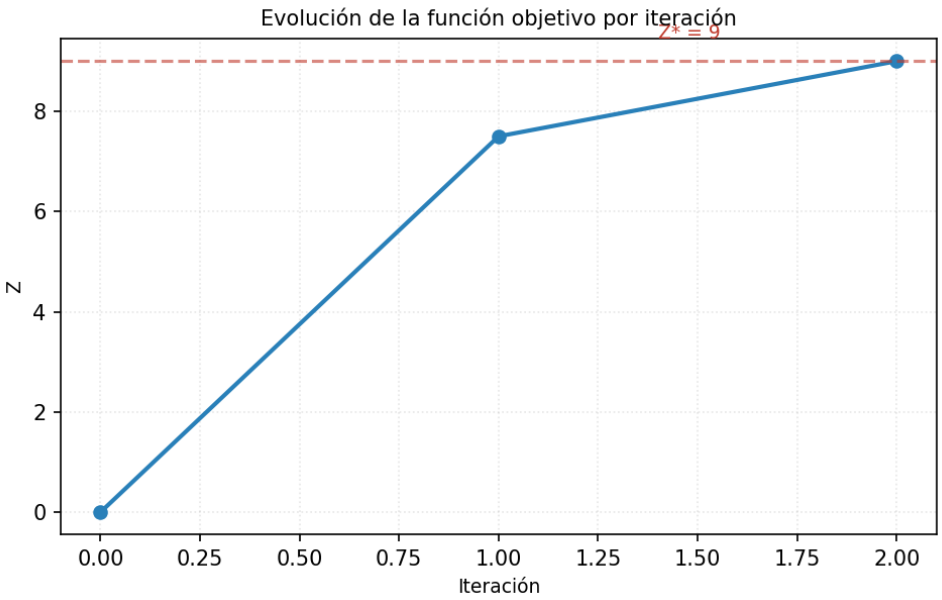
8.4 En gráficas

La región factible de este problema es el polígono con vértices (0,0), (2.5,0), (1,3) y (0,4). La línea de nivel de Z = 9 (más intensa) es la última que toca la región, en el vértice (1,3):



Región factible con el óptimo en rojo y la flecha de mejora.

Nota: como el método caminó (0,0) → (2.5,0) → (1,3), el progreso de Z fue 0 → 7.5 → 9, monótono creciente.



Z nunca baja: en maximización cada iteración mejora (o iguala) la solución hasta converger a 9.

9. Las facilidades de Python que hicieron esto simple

Vale la pena cerrar con el inventario de por qué escribir este método en Python (con NumPy) es notablemente más corto que en un lenguaje clásico:

dataclass — el modelo. Con **@dataclass** el problema, cada paso y el resultado se declaran en pocas líneas con tipos legibles (str | None) en vez de escribir getters, constructores y __repr__ a mano.

```
from dataclasses import dataclass
@dataclass
class SimplexStep:
    kind: str
    entering: str | None
```

Lectura de la fila objetivo como suma de productos. z_j = c_B · $columna_j$ es un producto punto; NumPy lo calcula para todas las columnas a la vez con @.

```
implied_costs = basic_costs @ canonical.matrix
```

Slicing del tableau. Con tableau[-1, :-1] se extrae la fila objetivo sin la última columna, y con matrix[row, -1] el lado derecho de cada fila. Sin bucles manuales.

```
objective_row = tableau[-1, :-1]
```

Operaciones por fila completas. Dividir una fila entre el pivote o restarle un múltiplo de otra es una línea porque NumPy opera sobre el vector completo.

```
matrix[row] = matrix[row] - factor * matrix[other_row]
```

min con key y tuplas (regla de Bland). Elegir el mejor candidato y al mismo tiempo desempatar por índice es una sola llamada:

```
min(candidates, key=lambda i: (objective_row[i], i))
```

Tolerancias para el punto flotante. Los ceros calculados suelen salir como 1e-17. Constantes con nombre distinguen 'es cero' (1e-9) de 'la artificial vale de verdad' (1e-6):

```
NEGLIGIBLE = 1e-9
INFEASIBLE_TOLERANCE = 1e-6
```

copy() para la trazabilidad. Cada paso guarda matrix=tableau.copy() para que el pivote siguiente no corrompa la historia mostrada al usuario.

```
matrix=tableau.copy()
```

Listas por comprensión y f-strings para las etiquetas. Generar los nombres x1…xn, s1…, e1…, a1… es una expresión:

```
labels = variable_labels + slack_labels + surplus_labels + artificial_labels_list
```

Listas por comprensión para contar. Contar cuántas columnas de cada tipo se necesitan sin un solo if:

```
slack_columns = sum(c.relation == '<=' for c in problem.constraints)
```

La heurística de M. El valor de Big-M se deriva de los propios datos para que siempre domine a los costos reales:

```
M = 1 + sum(abs(c)) + sum(abs(b)) + max(sum(abs(fila de A)))
```

10. Para cerrar: cómo usar el proyecto

La lógica de este documento vive en la carpeta **simplex/** del proyecto, separada de la interfaz:

```
simplex/problem.py   el modelo LPProblem
simplex/validate.py  validación de entrada
simplex/big_m.py     el algoritmo de este documento
explain/             glosario y textos de explicación
viz/plots.py         las gráficas
gui/                 la interfaz Tkinter
```

Para probarlo tú mismo:

• `pip install -r requirements.txt` — instala numpy y matplotlib.
• `python main.py` — abre la aplicación; eliges max/min, escribes las restricciones con ≤, ≥ o =, y cada iteración aparece con su tabla y su explicación.
• `python tests/test_simplex.py` — ejecuta las pruebas del algoritmo (el proyecto se validó contra scipy en más de 100 problemas aleatorios).

En la interfaz, la pestaña 'Paso a paso' muestra exactamente lo que se calcula en los capítulos 5 y 8: qué variable entra, el test del cociente con sus números reales, el pivote resaltado, y por qué se llega al óptimo.

Este documento se generó con el propio script docs/generate_pdf.py usando ReportLab, ejecutando el algoritmo real para producir los números del ejemplo.